High-Level Design (HLD) Document

Project: RAG-Based Customer Support Assistant

1. System Overview

- **Problem Definition:** Customer support teams face a high volume of repetitive queries that consume valuable human resources. Traditional chatbots often struggle with complex queries, lack specific domain knowledge, and are prone to hallucinating incorrect answers, which can frustrate users and damage brand trust.
- **Scope of the System:** This system aims to automate customer support by answering user queries accurately using a Retrieval-Augmented Generation (RAG) approach based on a proprietary PDF knowledge base. To ensure reliability, the system incorporates a graph-based workflow to evaluate its own confidence and gracefully escalate unanswerable or complex queries to a human agent via a Human-in-the-Loop (HITL) mechanism.

2. Architecture Diagram Representation

Note: For the PDF version, it is recommended to recreate this flow in a tool like Draw.io or Lucidchart.

1. **User Interface:** A Command Line Interface (CLI) or Web Chat UI takes the user's query.
2. **Document Ingestion Pipeline:** PDF documents are loaded and split into manageable chunks.
3. **Embedding System:** Text chunks are passed through an embedding model to generate vector representations.
4. **Vector Database:** Vectors and their metadata are stored for semantic search.
5. **Retrieval Layer:** The user query is embedded, and the database is queried for the most semantically similar text chunks.
6. **Workflow Orchestration (LangGraph):** The core control logic routes the query and context.
7. **LLM Processing Layer:** The Large Language Model evaluates intent and generates answers.
8. **HITL System:** A fallback mechanism triggered by the routing layer for manual intervention.

3. Component Description

- **Document Loader:** Parses raw text from the uploaded PDF knowledge base.
- **Chunking Strategy:** Splits the parsed text into smaller, overlapping segments (e.g., 1000 characters with 200 character overlap) to maintain semantic boundaries and fit within LLM context windows.
- **Embedding Model:** Converts text chunks into high-dimensional numerical vectors (e.g., HuggingFace embeddings or OpenAI text-embedding-3).
- **Vector Store:** A specialized database (ChromaDB) optimized for storing and querying vector embeddings.
- **Retriever:** The search engine that compares the user's embedded query against the vector store to fetch the top *k* most relevant chunks.

- **LLM:** The reasoning engine (e.g., Llama 3 or GPT-4) used for both evaluating intent and generating the final natural language response.
- **Graph Workflow Engine:** LangGraph acts as a state machine, orchestrating the sequence of operations (nodes) and passing a shared state object between them.
- **Routing Layer:** A conditional logic gate that decides whether the LLM should generate an answer or if the query should be escalated.
- **HITL Module:** A pause in the automated execution that alerts a human agent, provides the context, and allows them to manually input a response.

4. Data Flow

1. **Ingestion:** PDF -> Text Splitter -> Embedding Model -> Vector Database.
2. **Query Lifecycle:** User Query -> Embedding Model -> Retriever -> (Context + Query) -> LangGraph State.
3. **Evaluation:** Graph routes to the "Intent Node." The LLM evaluates if the context is sufficient to answer the query.
4. **Execution:** * If Intent = "Confident", route to "Generation Node" -> Output to User.
    - If Intent = "Uncertain", route to "HITL Node" -> Human Agent inputs response -> Output to User.

5. Technology Choices

- **ChromaDB:** Chosen for the vector database because it is lightweight, open-source, and runs seamlessly locally, making it ideal for rapid prototyping and deployment without heavy cloud infrastructure.
- **LangGraph:** Selected over standard linear chains (like standard LangChain pipelines) because it treats the application as a cyclic graph. This allows for complex state management and the necessary conditional routing required for the HITL fallback.
- **LLM Choice:** A fast-inference model (e.g., Llama 3 via Groq API or GPT-3.5-Turbo) is recommended to minimize latency during the two-step LLM process (Intent Evaluation + Generation).

6. Scalability Considerations

- **Handling Large Documents:** By chunking documents and only retrieving the top *k* results, the prompt size remains constant, allowing the system to scale to thousands of pages without breaking the LLM context limit.
- **Increasing Query Load:** For enterprise scale, the local ChromaDB can be swapped for a cloud-managed vector database (like Pinecone or Weaviate), and the application can be containerized using Docker.
- **Latency Concerns:** To reduce latency, a smaller, localized embedding model (like all-MiniLM-L6-v2) can be used, and asynchronous LLM calls can be implemented.